Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

Tutorial 6 : Pipeline instrumenté, cascade et fusion de décisions sur la Portenta H7 avec OpenMV

Séries de tuto sur Portenta H7, lecture 6 « Déployer, profiler et intégrer l'IA embarquée »

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

2 novembre 2026

Objectifs du Tutorial

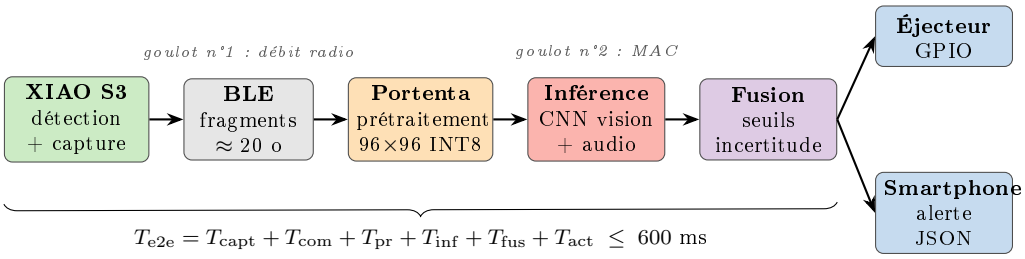
Au Tutorial 5, le modèle INT8 tournait seul, en boucle. Ici, vous le transformez en **brique d'un système** : vous vérifiez qu'il voit exactement ce qu'il a vu à l'entraînement, vous mesurez chaque étape du pipeline, vous ajoutez un étage de détection qui épargne le modèle, vous calibrez ses confiances, et vous l'enfermez dans une machine d'états qui produit des messages JSON. Tout se passe sur la seule Portenta ; le TP 6 y branchera la XIAO et le smartphone.
À la fin de ce tutoriel, vous serez capable de :

- Vérifier l'identité du prétraitement PC/carte avec une image de référence.
- Instrumenter un pipeline (horodatages et broche de sonde) et produire médiane, p95 et maximum par étape.
- Construire une cascade « détection de présence → classification » et mesurer son gain.
- Calibrer le modèle par *temperature scaling* et exporter une version qui sort les logits.
- Fusionner deux vues, appliquer un seuil de rejet et piloter le tout par une machine d'états.
- Estimer l'énergie par décision à partir de mesures de courant.

Durée indicative : 1 h 45. Pré-requis : Tutorials 1 et 5 terminés, fichiers `trash_int8.tflite`, `labels.txt` et notebook du Tutorial 5.

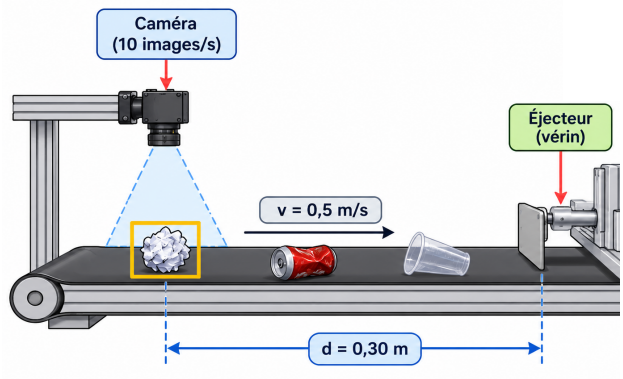
1 Introduction : du modèle au pipeline

Un modèle ne trie rien : c'est le **pipeline** qui trie. Entre l'objet sur le tapis et le coup d'éjecteur, il y a une capture, un prétraitement, une inférence, une décision et une action, et chaque maillon a une durée, une consommation et ses propres modes de défaillance.



La chaîne complète de la station. Dans ce tutoriel, la capture est faite par la Portenta elle-même (la liaison BLE avec la XIAO est l'objet du TP 6).

Ce que nous allons traiter

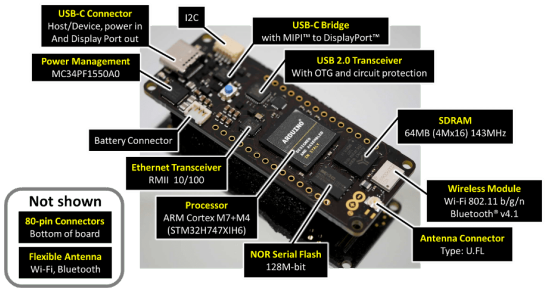


La station EcoTri 4.0 : caméra au-dessus du tapis, éjecteur en aval. Le temps disponible pour décider est fixé par la distance caméra-éjecteur et la vitesse du tapis.

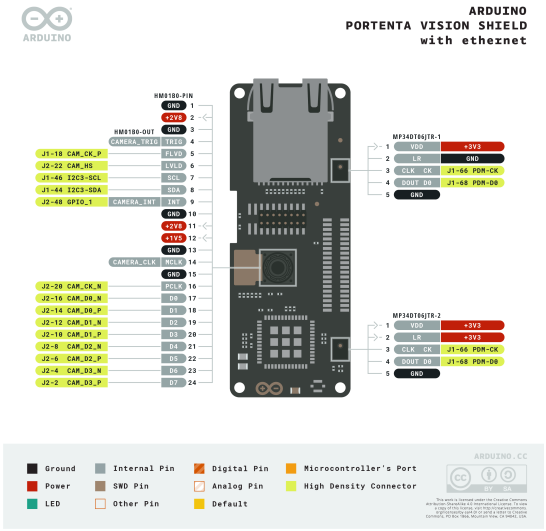
2 Pré-requis

Matériel

- Carte **Arduino Portenta H7** et **Vision Shield**, câble USB-C.
- Les objets du fil rouge (canettes, bouteilles, cartons, bocaux) et un fond uniforme.
- Optionnel : un **USB-mètre** (ou un module INA219) pour la mesure d'énergie de l'étape 8, un oscilloscope ou un analyseur logique pour la broche de sonde.



Portenta H7



Vision Shield : broches disponibles pour la sonde

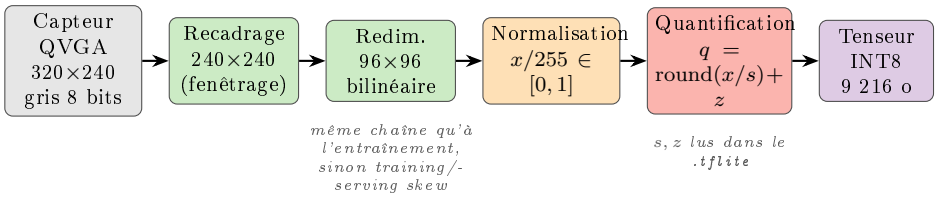
Logiciel

1. **OpenMV IDE 4.x**, firmware **4.5 ou plus** (module `ml`).
2. Le notebook **Colab** du Tutorial 5 (modèle `trash_mnv2.keras`, `val_ds`, fonction `rep_data`).

3 Étapes détaillées du tutoriel

Étape 1 : Le test de l'image de référence (prétraitement identique)

Le piège le plus coûteux d'un déploiement est silencieux : la carte ne prépare pas l'image comme le notebook. On le détecte avec une **image de référence** classée des deux côtés.



La chaîne de prétraitement à reproduire à l'identique. Vous allez comparer la sortie du modèle sur PC et sur la carte pour une même image.

a) Sur la carte, capturez une image et enregistrez-la ainsi que la sortie du modèle :

```
# T6-1 : image de référence et sortie du modèle sur la Portenta
import sensor, ml
sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA); sensor.set_windowing((240, 240))
sensor.set_auto_gain(False); sensor.set_auto_exposure(False, exposure_us=8000)
sensor.skip_frames(time=1500)

net = ml.Model("trash_int8.tflite", load_to_fb=True)
print(net)                                  # formes, types, s, z
img = sensor.snapshot()
```

```
img.save("ref.bmp")                              # sans compression
p = net.predict([img])[0].flatten().tolist()
print("carte :", ["%.3f" % v for v in p])
```

b) Dans Colab, téléversez `ref.bmp` et appliquez **votre** prétraitement d'entraînement :

```
from PIL import Image
im = Image.open("ref.bmp").convert("L").resize((96, 96), Image.BILINEAR)
x = np.asarray(im, np.float32)[None, :, :, None] / 255.0     # [0,1] comme au
    Tutorial 5
it = tf.lite.Interpreter(model_path="trash_int8.tflite"); it.allocate_tensors()
ie, oe = it.get_input_details()[0], it.get_output_details()[0]
s, z = ie["quantization"]
it.set_tensor(ie["index"], np.clip(np.round(x / s + z), -128, 127).astype(np.int8))
it.invoke()
so, zo = oe["quantization"]
print("PC    :", np.round(so * (it.get_tensor(oe["index"])[0] - zo), 3))
```

1. Les deux vecteurs doivent coïncider à $\pm 0{,}02$ près (l'écart résiduel vient de l'interpolation).
2. Modifiez volontairement la normalisation dans Colab (`x * 2 - 1`) : comment évolue la sortie ? C'est exactement ce que subirait la station avec un prétraitement faux.

Question

Pourquoi enregistrer en BMP plutôt qu'en JPEG pour ce test ? Pourquoi fige-t-on le gain et l'exposition ?

Étape 2 : Un profileur réutilisable

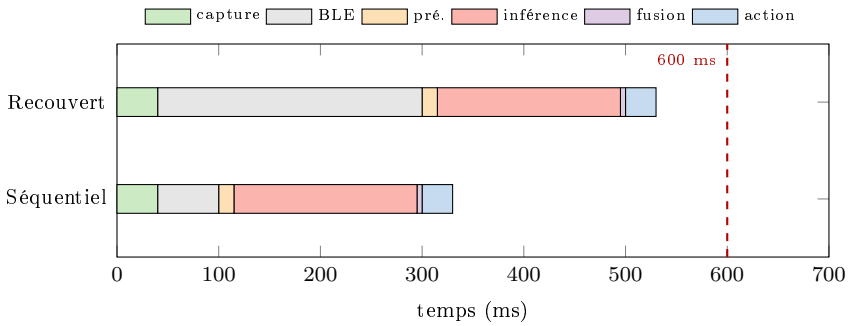
Créez sur la carte le fichier `profil.py` (il servira au TP) :

```
# profil.py -- horodatage des étapes et statistiques (médiane, p95, max)
import time
from pyb import Pin

class Profil:
    def __init__(self, etapes, broche="D5"):
        self.etapes = etapes
        self.t = {e: [] for e in etapes}
        self.sonde = Pin(broche, Pin.OUT_PP)      # bascule à chaque étape
    def top(self, e):
        self.sonde.value(not self.sonde.value())
        self.t[e].append(time.ticks_us())
    def stats(self):
        res = {}
        for a, b in zip(self.etapes, self.etapes[1:]):
            d = sorted(time.ticks_diff(y, x) / 1000 for x, y in zip(self.t[a], self.t
    [b]))
            n = len(d)
            if n: res[b] = (d[n // 2], d[min(n - 1, int(.95 * n))], d[-1])
        return res
    def rapport(self):
        tot = 0
        for e, (med, p95, mx) in self.stats().items():
            tot += med
            print("%-9s med %7.1f | p95 %7.1f | max %7.1f ms" % (e, med, p95, mx))
        print("TOTAL (somme des médianes) %.1f ms" % tot)
```

Puis mesurez le pipeline simple (capture, inférence, décision, affichage) sur 100 objets :

```
# T6-2 : profil du pipeline de base
import sensor, ml, gc
from profil import Profil
# ... (initialisation du capteur et du modèle comme à l'étape 1)
P = Profil(["debut", "capture", "inference", "decision"])
labels = [l.strip() for l in open("labels.txt")]
for _ in range(5): net.predict([sensor.snapshot()])      # échauffement
gc.collect()
for i in range(100):
    P.top("debut")
    img = sensor.snapshot();                          P.top("capture")
    p = net.predict([img])[0].flatten().tolist(); P.top("inference")
    k = p.index(max(p)); d = labels[k] if p[k] > 0.7 else "doute"
    P.top("decision")
P.rapport()
print("tas libre :", gc.mem_free())
```



Reportez vos médianes sous cette forme (barres empilées). Ici, sans radio, l'inférence devrait dominer.

1. Exécutez une fois avec le **Frame Buffer de l'IDE activé**, une fois désactivé. Quelle étape change, et de combien ?
2. Retirez le `gc.collect()` et ajoutez dans la boucle une allocation inutile (`s = str(p)`). Le maximum de quelle étape augmente ? Pourquoi ?
3. Si vous disposez d'un oscilloscope sur `D5`, comparez la largeur des créneaux aux valeurs imprimées.

Étape 3 : Estimer le rendement η

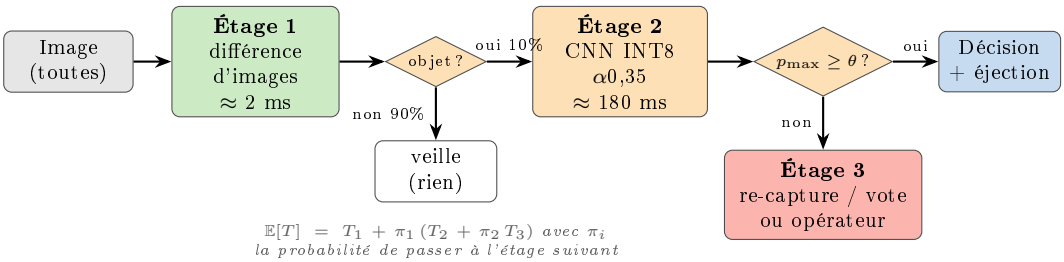
Avec $N_{\text{MAC}} \approx 11$ M (MobileNetV2 α0,35, 96×96) et $f = 480$ MHz, calculez $\eta = N_{\text{MAC}}/(T_{\text{inf}} f)$ à partir de votre médiane. Relancez avec `trash_fp32.tflite` du Tutorial 5 (s'il se charge) et comparez les deux η.

Question

Le rapport des latences float32/INT8 est-il proche de 4 ? Citez deux raisons pour lesquelles il est en général **plus grand**.

Étape 4 : Étage 1 de la cascade, la détection de présence

Sur un tapis, la plupart des images sont vides. Inutile de réveiller le CNN pour elles.



Vous implémentez ici les étages 1 et 2 ; l'étage 3 (re-capture) arrive à l'étape 7.

```
# T6-4 : cascade différence d'images -> CNN
import sensor, ml, time, gc
from profil import Profil
sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA); sensor.set_windowing((240, 240))
sensor.set_auto_gain(False); sensor.set_auto_exposure(False, exposure_us=8000)
sensor.skip_frames(time=1500)
net = ml.Model("trash_int8.tflite", load_to_fb=True)

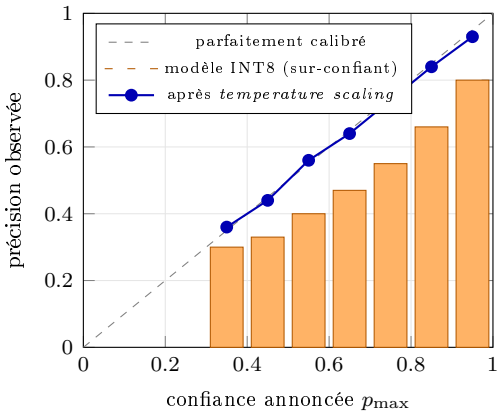
print("Laissez le tapis VIDE...")
sensor.skip_frames(time=1000)
fond = sensor.snapshot().copy()                    # image de fond (copie en RAM)
SEUIL = 6                                          # à régler (question 1)

def objet_present(img):
    tmp = img.copy()
    tmp.difference(fond)                           # |img - fond| pixel par pixel
    return tmp.get_statistics().mean() > SEUIL

n_img = n_cnn = 0; t1 = []; t2 = []
t_fin = time.ticks_add(time.ticks_ms(), 30000)     # 30 s d'essai
while time.ticks_diff(t_fin, time.ticks_ms()) > 0:
    img = sensor.snapshot(); n_img += 1
    a = time.ticks_us(); present = objet_present(img); t1.append(time.ticks_diff(time
    .ticks_us(), a))
    if present:
        a = time.ticks_us(); net.predict([img]); t2.append(time.ticks_diff(time.
    ticks_us(), a)); n_cnn += 1
pi1 = n_cnn / n_img
T1 = sum(t1) / len(t1) / 1000; T2 = (sum(t2) / len(t2) / 1000) if t2 else 0
print("pi1 = %.2f | T1 = %.1f ms | T2 = %.1f ms | E[T] = %.1f ms (contre %.1f)" %
      (pi1, T1, T2, T1 + pi1 * T2, T2))
```

1. Réglez `SEUIL` : affichez `get_statistics().mean()` tapis vide puis avec un objet, et placez le seuil entre les deux, plus près du « vide » (on privilégie le rappel).
2. Pendant les 30 s, posez un objet environ une fois sur cinq. Relevez π_1, T_1, T_2 et $\mathbb{E}[T]$. Vérifiez la formule du cours.
3. Allumez une lampe à côté du banc : que devient π_1 ? Proposez une mise à jour lente du fond (indice : `fond.blend(img, alpha=...)` quand aucun objet n'est présent).

Étape 5 : Calibrer les confiances dans Colab



Vous allez tracer ce diagramme pour votre modèle, avant et après *temperature scaling*.

```
# Logits = sortie de l'avant-dernière couche (avant softmax)
from tensorflow.keras import Model, layers
dense = model.layers[-1]
logit_model = Model(model.input, layers.Dense(dense.units, name="logits")(dense.input
    ))
logit_model.layers[-1].set_weights(dense.get_weights())

Z = np.concatenate([logit_model.predict(x, verbose=0) for x, _ in val_ds])
Y = np.concatenate([y.numpy() for _, y in val_ds])

def nll(T):
    z = Z / T; z = z - z.max(1, keepdims=True)
    lp = z - np.log(np.exp(z).sum(1, keepdims=True))
    return -lp[np.arange(len(Y)), Y].mean()

def ece(T, B=10):
    z = Z / T; p = np.exp(z - z.max(1, keepdims=True)); p /= p.sum(1, keepdims=True)
    conf, pred = p.max(1), p.argmax(1); e = 0
    for b in range(B):
        m = (conf > b / B) & (conf <= (b + 1) / B)
        if m.any(): e += m.mean() * abs((pred[m] == Y[m]).mean() - conf[m].mean())
    return e

Ts = np.linspace(0.5, 4, 71); T = Ts[np.argmin([nll(t) for t in Ts])]
print(f"T optimal = {T:.2f} | ECE avant {ece(1.0):.3f} -> après {ece(T):.3f}")
```

Exportez ensuite le modèle des logits en INT8 (même fonction `convertir` qu'au Tutorial 5, avec `logit_model`) sous le nom `trash_logits_int8.tflite`, et notez `T` : la carte appliquera softmax(z/T) elle-même.

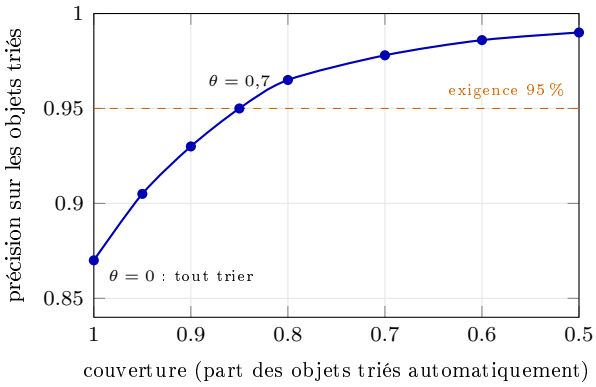
Question

Le temperature scaling change-t-il la précision du modèle ? Change-t-il le nombre d'objets rejetés pour un seuil $\theta = 0{,}7$ donné ? Expliquez.

Étape 6 : Choisir le seuil de rejet par le coût

```
z = Z / T; P = np.exp(z - z.max(1, keepdims=True)); P /= P.sum(1, keepdims=True)
conf, pred = P.max(1), P.argmax(1)
```

```
C_ERR, C_REJ = 5, 1                                  # coûts relatifs (à discuter en é
    quipe)
for th in [0.0, 0.5, 0.6, 0.7, 0.8, 0.9]:
    ok = conf >= th
    err = int(((pred != Y) & ok).sum()); rej = int((~ok).sum())
    prec = (pred[ok] == Y[ok]).mean() if ok.any() else float("nan")
    print(f"theta={th:.1f} couverture={ok.mean():.2f} précision={prec:.3f} "
          f"err={err:3d} rej={rej:3d} coût={C_ERR*err + C_REJ*rej}")
```



Tracez votre propre courbe précision-couverture ; le seuil retenu est celui qui minimise le coût.

Étape 7 : Fusion de deux vues et machine d'états

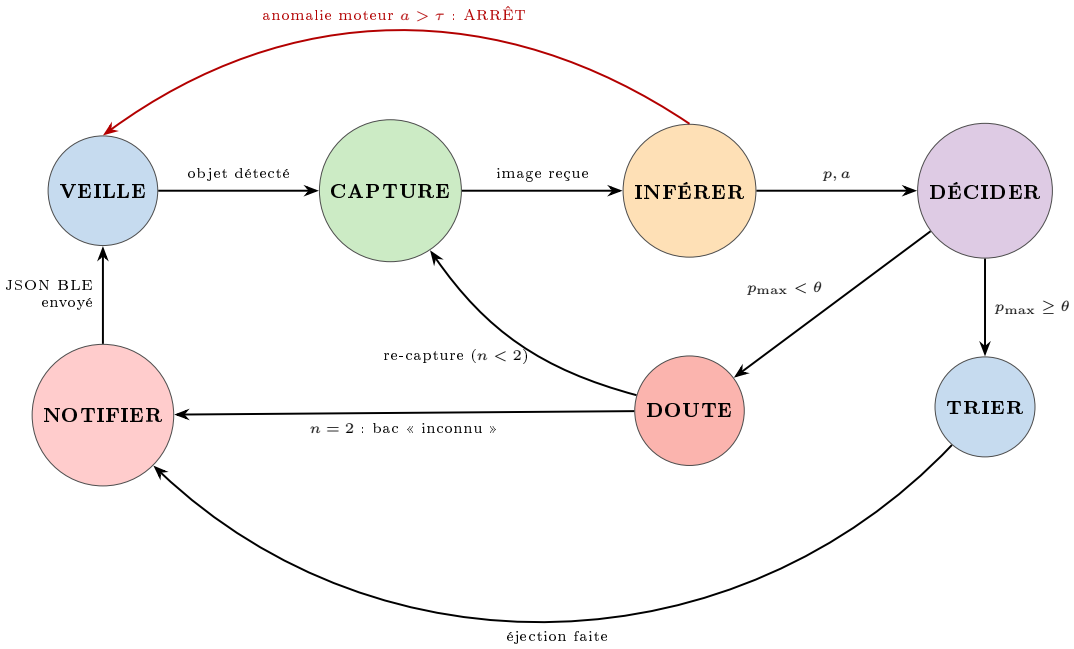
Copiez `trash_logits_int8.tflite` sur la carte et créez `decision.py` :

```
# decision.py -- softmax tempéré, fusion multi-vues, règles prioritaires
import math
T_CAL, THETA, TAU = 1.8, 0.7, 0.8                # vos valeurs (étapes 5 et 6)

def softmax_t(z, T=T_CAL):
    m = max(z); e = [math.exp((v - m) / T) for v in z]; s = sum(e)
    return [v / s for v in e]

def fusion_log(vues):                            # règle du produit, en log
    s = [0.0] * len(vues[0])
    for p in vues:
        for c, pc in enumerate(p): s[c] += math.log(max(pc, 1e-4))
    return softmax_t(s, 1.0)

def decider(p, a=0.0):
    if a > TAU: return "ARRET", None
    k = p.index(max(p))
    return ("TRIER" if p[k] >= THETA else "DOUTE"), k
```



La machine d'états que vous implémentez ; l'état audio est simulé ici par `a = 0` (le TP 6 branchera le modèle de la lecture 4).

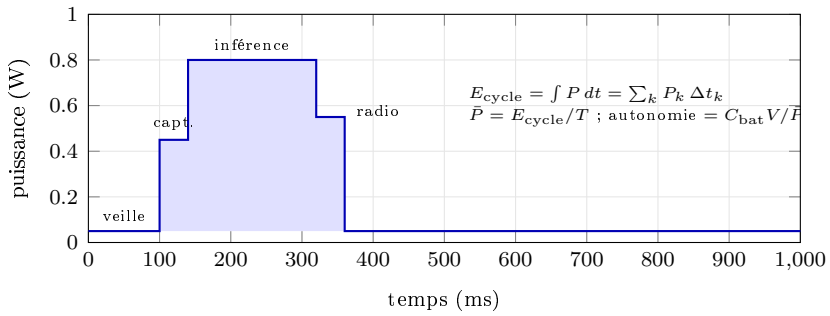
```
# T6-7 : machine d'états de l'unité de décision (version mono-carte)
import sensor, ml, time, gc, json
from pyb import LED
from decision import softmax_t, fusion_log, decider
# ... initialisation du capteur, fond et objet_present() comme à l'étape 4
net = ml.Model("trash_logits_int8.tflite", load_to_fb=True)
labels = [l.strip() for l in open("labels.txt")]
VEILLE, CAPTURE, INFERER, DECIDER, TRIER, DOUTE, NOTIFIER = range(7)
etat, vues, nid, t0 = VEILLE, [], 0, 0

while True:
    if etat == VEILLE:
        img = sensor.snapshot()
        if objet_present(img): t0 = time.ticks_ms(); etat = INFERER
    elif etat == CAPTURE:                          # nouvelle vue du même objet
        time.sleep_ms(150); img = sensor.snapshot(); etat = INFERER
    elif etat == INFERER:
        z = net.predict([img])[0].flatten().tolist()
        vues.append(softmax_t(z)); etat = DECIDER
    elif etat == DECIDER:
        d, k = decider(fusion_log(vues), a=0.0)
        etat = {"TRIER": TRIER, "DOUTE": DOUTE, "ARRET": NOTIFIER}[d]
    elif etat == DOUTE:
        etat = CAPTURE if len(vues) < 2 else NOTIFIER
    elif etat in (TRIER, NOTIFIER):
        p = fusion_log(vues); k = p.index(max(p))
        msg = {"v": 1, "id": nid, "etat": "TRIER" if etat == TRIER else "INCONNU",
               "classe": labels[k], "p": round(p[k], 2), "vues": len(vues),
               "lat": time.ticks_diff(time.ticks_ms(), t0), "mdl": "mnv2a035-int8-cal
    "}
        LED(2 if etat == TRIER else 1).on(); print(json.dumps(msg))
        time.sleep_ms(300); LED(1).off(); LED(2).off()
```

```
        nid += 1; vues = []; gc.collect()
        while objet_present(sensor.snapshot()): pass    # attendre que l'objet parte
        etat = VEILLE
```

1. Présentez une dizaine d'objets, dont certains difficiles (bouteille froissée, carton brillant). Combien passent par l'état DOUTE ? Combien sont rattrapés par la deuxième vue ?
2. Lisez dans le JSON la latence `lat` : distinguez les objets décidés en une vue et en deux vues.
3. Où placeriez-vous un chien de garde (`machine.WDT`) et avec quel délai ?

Étape 8 : Énergie par décision



Profil de puissance attendu ; l'énergie par décision est l'aire sous la courbe pendant un cycle.

1. Avec un USB-mètre, relevez le courant moyen (a) tapis vide (étage 1 seul), (b) inférence en continu (modifiez la boucle pour classer chaque image). Tension : 5 V.
2. Déduisez $P_{\text{veille active}}$ et P_{inf}, puis l'énergie par décision $E \approx P_{\text{inf}} \times T_{\text{inf}}$.
3. Calculez la puissance moyenne pour un objet toutes les 2 s, avec et sans cascade. Quel gain relatif ?

Astuce

La Portenta n'a pas de mesure de courant intégrée. Sans USB-mètre, utilisez les valeurs typiques du cours et concentrez-vous sur le **rapport** des temps : c'est lui qui détermine le gain de la cascade.

4 Pour aller plus loin

— C. Guo et al. (Cornell), « On Calibration of Modern Neural Networks », ICML 2017 : arxiv.org/abs/1706.04599.

— MLPerf Tiny (Harvard, MLCommons) : méthode de mesure de latence et d'énergie, mlcommons.org.

— Harvard *Machine Learning Systems*, chapitre « Benchmarking AI » : mlsysbook.ai.

— Documentation OpenMV, modules `ml`, `image` (`difference`, `blend`) et `machine` (`WDT`, `lightsleep`) : docs.openmv.io.